**VIỆN NGHIÊN CỨU & ĐÀO TẠO VIỆT - ANH (VNUK)**

**ĐẠI HỌC ĐÀ NẴNG**

# BÁO CÁO TỔNG KẾT TOÀN DIỆN ĐỒ ÁN CƠ SỞ

## NGHIÊN CỨU, THIẾT KẾ & HIỆN THỰC HÓA HỆ THỐNG GOMOKU AI ĐA TẦNG: TỪ HEURISTIC MINIMAX ĐẾN ALPHAZERO HỌC TĂNG CƯỜNG

**(Master Thesis Synthesis & Comprehensive Defense Dossier)**

**Sinh viên thực hiện:** Nguyễn Tâm Đức
**Mã sinh viên:** 23010405
**Ngành đào tạo:** Khoa học Dữ liệu & Trí tuệ Nhân tạo (DS-AI)

Đà Nẵng, Tháng 11 Năm 2026

**Abstract**

Báo cáo chuyên khảo này là công trình tổng kết toàn diện nhất của Đồ án Cơ sở, tích hợp trọn vẹn toàn bộ các giai đoạn nghiên cứu từ nền tảng thuật toán cổ điển đến các mô hình học sâu và học tăng cường hiện đại. Đồ án xây dựng một hệ thống cờ Gomoku AI đối kháng $15 \times 15$ hoàn chỉnh qua 5 cột mốc: (1) Core Engine Heuristic & Cắt tỉa Alpha-Beta kết hợp bảng băm Zobrist 64-bit; (2) Pipeline chuẩn hóa dữ liệu lớn trích xuất từ 10,000 ván đấu ($368,401$ mẫu) với đại số nhóm đối xứng Dihedral $D_4$; (3) Khởi động ấm mạng nơ-ron Dual-Head ResNet-6 trên phần cứng GPU NVIDIA T1200 đạt độ chính xác Top-3 $82.02$%; (4) Công cụ tìm kiếm cây Monte Carlo có định hướng nơ-ron (AlphaZero MCTS) hạ gục bot Heuristic chỉ sau 13 nước đi; và (5) Vòng lặp cải tiến chính sách tự chơi khép kín (Self-Play Policy Iteration) với cổng kiểm định nhà vô địch (Champion Gating).

Đặc biệt, tài liệu trang bị phần cẩm nang gồm **15 câu hỏi phản biện học thuật hóc búa nhất** của Hội đồng bảo vệ đồ án, cung cấp đầy đủ luận cứ toán học, số liệu đo đạc thực nghiệm và phân tích trade-off chuẩn mực giúp sinh viên tự tin bảo vệ xuất sắc.

# Contents

**1 Tổng Quan Đồ Án & Không Gian Trạng Thái Gomoku** **2**
1.1 Bối Cảnh Nghiên Cứu & Bài Toán Cờ Caro 15x15 . . . . . 2
1.2 Bản Đồ 5 Chặng Đường Phát Triển Khoa Học Của Đồ Án . . . . 2

**2 Ma Trận So Sánh Năng Lực Toàn Diện Giữa Các Thế Hệ Bot** **4**

**3 Cẩm Nang 15 Câu Hỏi Phản Biện Hội Đồng & Lời Giải Điểm 10/10** **5**

**4 Kết Luận & Bàn Giao Sản Phẩm Đồ Án** **9**

**Tài Liệu Tham Khảo** **10**

# Chapter 1

# Tổng Quan Đồ Án & Không Gian Trạng Thái Gomoku

## 1.1 Bối Cảnh Nghiên Cứu & Bài Toán Cờ Caro 15x15

Cờ Gomoku (Caro) trên bàn cờ $15 \times 15$ là bài toán tìm kiếm đối kháng thông tin hoàn hảo (Perfect Information Two-Player Zero-Sum Game). Không gian trạng thái khả dĩ của bàn cờ:

$$|\mathcal{S}| \approx 3^{225} \approx 10^{107} \tag{1.1}$$

và hệ số phân nhánh trung bình $b \approx 30 - 40$. Việc vét cạn bằng thuật toán Minimax truyền thống chỉ có thể vươn tới độ sâu tối đa 3–4 ply trong giới hạn thời gian thực. Do đó, việc chuyển đổi từ Heuristic cổ điển sang mô hình Trực giác Nơ-ron kết hợp Cây MCTS là bước đi mang tính tất yếu khoa học.

## 1.2 Bản Đồ 5 Chặng Đường Phát Triển Khoa Học Của Đồ Án

1. **Baseline Cổ Điển (Tuần 1–10):** Xây dựng Game Engine bằng TypeScript và Rust, cài đặt lượng giá mẫu Heuristic, cắt tỉa Alpha-Beta, sắp xếp nước đi và bảng băm Zobrist.
2. **Phase 1 (Data Pipeline):** Trích xuất 10,000 ván đấu thực nghiệm $\rightarrow 368,401$ transitions, mã hóa tensor 3 mặt phẳng $3 \times 15 \times 15$, đối xứng nhóm nhị diện $D_4$ ($8\times \implies 2.94$ triệu biến thể), phân chia không rò rỉ (Zero-Leakage Split).
3. **Phase 2 (Behavioral Cloning):** Huấn luyện mạng Dual-Head ResNet-6 trên GPU NVIDIA T1200 Laptop, tăng tốc AMP FP16, Tensor Cores, đạt Top-1 $60.06$%, Top-3 $82.02$%.

4. **Phase 3 (Neural MCTS):** Tích hợp cây Monte Carlo với công thức PUCT, bơm nhiễu Dirichlet Dir$(0.03)$, triệt tiêu nước đi sai sót (Blunder Elimination), thắng áp đảo Heuristic 3-0 chỉ sau 13 nước.
5. **Phase 4 (Self-Play RL):** Chu trình tự chơi tiến hóa đa thế hệ, Replay Buffer trượt, Cổng thẩm định vô địch Champion Gate ($\geq 55\%$), xuất mô hình ONNX `champion_ultimate.onnx`.

# Chapter 2

# Ma Trận So Sánh Năng Lực Toàn Diện Giữa Các Thế Hệ Bot

Table 2.1: Ma trận so sánh kỹ thuật giữa các thế hệ AI trong đồ án.

| **Tiêu chí so sánh** | **Heuristic Minimax** | **ResNet Pure** | **AlphaZero MC** |
|---|---|---|---|
| Cơ chế cốt lõi | Vét cạn Alpha-Beta | Mạng nơ-ron sâu | MCTS + ResN |
| Thời gian tính toán | $20 - 35$ ms | **1.5** ms | $40$ ms |
| Độ sâu tương đương | 2–3 ply cứng | Không duyệt | 6–12 ply linh h |
| Số lượng nút duyệt | $500 - 3,000$ | **1** forward pass | $40 - 50$ mô phỏ |
| Nguồn tri thức | Con người viết luật | 10,000 ván đấu | Trực giác + C |
| Tỷ lệ thắng khi cầm Tiên | $50\%$ | $50\%$ | **100**% (13 plie |
| Khả năng mở rộng | Rất thấp | Trung bình | Rất cao |

# Chapter 3

# Cẩm Nang 15 Câu Hỏi Phản Biện Hội Đồng & Lời Giải Điểm 10/10

Dưới đây là 15 câu hỏi phản biện hóc búa nhất mà Hội đồng chấm đồ án có thể đặt ra cho sinh viên, kèm theo phương pháp trả lời chặt chẽ, đanh thép:

**Câu hỏi Phản biện Hội đồng: 1. Tại sao bàn cờ Caro lại mã hóa 3 mặt phẳng mà không dùng ma trận số nguyên $\{0, 1, 2\}$?**

**Trả lời:** Nếu dùng số nguyên $\{0, 1, 2\}$, mạng nơ-ron tích chập sẽ hiểu nhầm đây là một đại lượng có thứ tự số học liên tục ($2 > 1 > 0$), trong khi giá trị ô cờ là các biến định danh phân loại rời rạc (Categorical). Hơn nữa, việc mã hóa 3 mặt phẳng bất biến phối cảnh (Mặt 0: Quân ta, Mặt 1: Quân địch, Mặt 2: Cờ màu Đen) cho phép mạng nơ-ron dùng chung một trọng số duy nhất cho cả hai người chơi, tăng gấp đôi hiệu quả mẫu.

**Câu hỏi Phản biện Hội đồng: 2. Làm thế nào em chứng minh được việc chia tập Train/Val không bị rò rỉ dữ liệu (Data Leakage)?**

**Trả lời:** Trong các trò chơi đối kháng, các trạng thái cờ trong cùng một ván đấu có tính tự tương quan thời gian (Temporal Autocorrelation) cực kỳ cao. Nếu chia ngẫu nhiên từng thế cờ, thế cờ ở lượt $t$ có thể nằm ở Train trong khi lượt $t + 1$ nằm ở Val, khiến độ chính xác validation bị ảo tưởng lên tới $99\%$. Em đã phân chia ở cấp độ ván cờ (**Match-Level Split**): 9,000 ván chỉ dùng cho Train và 1,000 ván độc lập chỉ dùng cho Val, bảo đảm rò rỉ quỹ đạo chính xác $0.0\%$.

**Câu hỏi Phản biện Hội đồng: 3. Tại sao mạng nơ-ron của em dùng cấu trúc Dual-Head thay vì 2 mạng riêng biệt cho Policy và Value?**

**Trả lời:** Sử dụng chung thân mạng ResNet-6 giúp hai nhiệm vụ chia sẻ biểu diễn đặc trưng không gian (Shared Feature Representation). Khả năng nhận diện hình cờ tốt giúp ích cho cả việc chọn nước đi lẫn lượng giá thế cờ. Hơn nữa, kiến trúc Dual-Head chỉ tốn đúng 1 lượt truyền tiến (Single Forward Pass) trên GPU, giảm $50\%$ tải tính toán so với việc chạy 2 mạng độc lập.

**Câu hỏi Phản biện Hội đồng: 4. Skip Connection trong ResNet giải quyết vấn đề gì về mặt toán học?**

**Trả lời:** Skip Connection biến đầu ra thành $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l)$. Khi lan truyền ngược, đạo hàm theo đầu vào là $\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_{l+1}}(\mathbf{I} + \mathbf{J}_{\mathcal{F}})$. Nhờ ma trận đơn vị $\mathbf{I}$, gradient luôn có một con đường cao tốc chảy ngược không suy giảm xuyên qua tất cả các lớp, triệt tiêu hoàn toàn hiện tượng suy biến gradient (Vanishing Gradient).

**Câu hỏi Phản biện Hội đồng: 5. Kỹ thuật Automatic Mixed Precision (AMP) hoạt động ra sao và tại sao cần GradScaler?**

**Trả lời:** AMP tính toán tích chập bằng số thực nửa độ chính xác FP16 (16-bit) trên Tensor Cores để tăng gấp đôi băng thông bộ nhớ. Tuy nhiên, dải biểu diễn FP16 nhỏ nên gradient dễ bị làm tròn về $0$ (Underflow). `GradScaler` khắc phục bằng cách nhân hàm mất mát với một hệ số lớn $S = 65,536$ trước khi backward, đẩy gradient vào vùng an toàn, sau đó thu nhỏ lại $1/S$ trước khi optimizer cập nhật trọng số.

**Câu hỏi Phản biện Hội đồng: 6. Trong công thức PUCT của MCTS, vai trò của hệ số $c_{\text{puct}}$ là gì?**

**Trả lời:** $c_{\text{puct}}$ là tham số điều phối sự cân bằng giữa Khai thác ($Q$) và Khám phá ($U$). Nếu $c_{\text{puct}}$ quá nhỏ, MCTS sẽ rơi vào bẫy tham lam cục bộ, chỉ duyệt các nước đi mạng gợi ý ban đầu. Nếu $c_{\text{puct}}$ quá lớn, MCTS sẽ thám hiểm dàn trải làm loãng số lần thăm. Trong đồ án, em thiết lập $c_{\text{puct}} = 2.0$ kết hợp giảm chấn mẫu $\frac{1}{1+N(s,a)}$.

**Câu hỏi Phản biện Hội đồng: 7. Tại sao phải bơm nhiễu Dirichlet $\text{Dir}(0.03)$ vào nút gốc của cây MCTS?**

**Trả lời:** Mạng nơ-ron có tính tất định cao. Nếu không có nhiễu ngẫu nhiên, khi tự chơi bot sẽ luôn đi một khai cuộc duy nhất. Phân phối Dirichlet với tham số $\alpha = 0.03 \approx 10/|A|$ có tính chất thưa thớt (sparsity), giúp tạo đột biến xác suất ngẫu nhiên vào một vài ô cờ bất ngờ, buộc

MCTS phải thám hiểm các thế trận mới lạ để tích lũy dữ liệu phong phú.

**Câu hỏi Phản biện Hội đồng: 8. Cờ Caro có tính chất sát thương 1-ply (Sudden Death), MCTS xử lý thế nào để không bị sơ suất?**

**Trả lời:** Em đã cài đặt cơ chế **Tactical Move Retention** trước khi duyệt: Nếu bản thân có nước nối 5 quân $\Longrightarrow$ Đi ngay lập tức để thắng trong $0.1$ ms; nếu đối thủ đe dọa tạo 5 quân ở lượt tới $\Longrightarrow$ Cây tìm kiếm thu hẹp phạm vi nút gốc chỉ duy nhất vào các ô chặn bắt buộc này. Nhờ đó, bot triệt tiêu $100\%$ các sơ suất ngớ ngẩn (Zero Blunder Policy).

**Câu hỏi Phản biện Hội đồng: 9. Tại sao trong Phase 4, kết quả tự chơi Iteration 1 lại bị từ chối (REJECTED)? Điều đó có phải là lỗi không?**

**Trả lời:** Hoàn toàn không phải lỗi mà là minh chứng cho sự vận hành chuẩn xác của **Champion Gating** theo đúng thiết kế của AlphaGo Zero. Ở Iter 1, chỉ với 886 mẫu, mạng mới chưa đủ mạnh và bị nhà vô địch Gen 0 đánh bại. Cổng Gating đã đóng vai trò van một chiều an toàn: Bác bỏ mô hình non nớt, giữ nguyên Gen 0, bảo đảm hệ thống có tính chất cải tiến đơn điệu (Monotonic Improvement Guarantee).

**Câu hỏi Phản biện Hội đồng: 10. Tại sao mạng nơ-ron của em chỉ chọn 6 blocks ResNet và 64 kênh mà không dùng 20 blocks như AlphaZero?**

**Trả lời:** Đây là bài toán tối ưu hóa tài nguyên phần cứng (Resource Economics). Bàn cờ Caro $15 \times 15$ có kích thước nhỏ hơn cờ vây ($19 \times 19$). Với 6 blocks (12 lớp Conv 3x3), trường tiếp nhận hữu hiệu (Effective Receptive Field) đạt $2 \times 6 \times 2 + 3 = 27$ ô, bao trùm toàn bộ đường kính 15 ô của bàn cờ. Mô hình chỉ có $2.53$ triệu tham số, tiêu tốn chưa đến $512$ MB VRAM, hoàn toàn chạy mượt mà theo thời gian thực ($40$ ms/nước) trên GPU laptop tầm trung NVIDIA T1200.

**Câu hỏi Phản biện Hội đồng: 11. Nhóm đối xứng Dihedral $D_4$ được ứng dụng như thế nào và vì sao lại tiết kiệm I/O?**

**Trả lời:** Bàn cờ vuông có tính chất bất biến qua 4 góc quay ($0°, 90°, 180°, 270°$) và 4 phép lật, tạo thành nhóm đối xứng $D_4$ cấp 8. Thay vì nhân $8\times$ kích thước file lưu trên đĩa (tốn hàng chục GB và nghẽn đọc đĩa), em áp dụng kỹ thuật **On-The-Fly Augmentation**: File nén trên đĩa chỉ $1.61$ MB, và mỗi khi nạp một batch vào GPU, thuật toán tự động chọn ngẫu nhiên 1 trong 8 phép biến đổi ma trận trong $0.05$ ms.

**Câu hỏi Phản biện Hội đồng: 12. Em hãy giải thích sự khác biệt giữa thuật toán Adam và AdamW trong kiến trúc có Batch Normalization?**

**Trả lời:** Trong Adam truyền thống, suy giảm trọng số (Weight Decay) được gộp chung vào gradient làm nhiễu véc-tơ động lượng thứ nhất và thứ hai. Đặc biệt khi có BatchNorm, việc scale trọng số bị chuẩn hóa triệt tiêu khiến L2 Regularization truyền thống mất đi tác dụng làm mượt độ phức tạp mô hình. AdamW bóc tách (decoupled) suy giảm trọng số trực tiếp khỏi gradient ($\theta_{t+1} = \theta_t - \eta\lambda\theta_t - \dots$), giúp kiểm soát độ lớn trọng số độc lập và hiệu quả hơn hẳn.

**Câu hỏi Phản biện Hội đồng: 13. Tại sao trong giải đấu Phase 3, AlphaZero MCTS lại thắng áp đảo Heuristic 3-0 chỉ sau 13 nước đi khi cầm quân Đen?**

**Trả lời:** Khi cầm quân Đen đi trước, AlphaZero MCTS sử dụng trực giác của mạng nơ-ron kết hợp mô phỏng PUCT để nhận diện các điểm liên kết đa hướng. Thay vì phòng ngự thụ động, MCTS liên tục tung ra các đòn tấn công gài bẫy dọc theo cột F. Do bot Heuristic Minimax chỉ duyệt sâu 2-ply cứng nên không lường trước được chuỗi bẫy sâu 6-8 ply, dẫn tới vỡ trận nhanh chóng.

**Câu hỏi Phản biện Hội đồng: 14. Mô hình ONNX đã xuất bản có thể triển khai trên những môi trường nào?**

**Trả lời:** Chuẩn mở ONNX (Open Neural Network Exchange) Opset 14 cho phép mô hình chạy độc lập hoàn toàn mà không cần cài đặt PyTorch. Nó có thể được nạp trực tiếp vào: (1) Trình duyệt Web qua ONNX Runtime Web / WebAssembly; (2) Engine Rust backend thông qua crate `ort`; (3) Ứng dụng Desktop qua khung Tauri; hoặc (4) Ứng dụng di động iOS/Android.

**Câu hỏi Phản biện Hội đồng: 15. Hướng phát triển tiếp theo của đồ án là gì?**

**Trả lời:** Đồ án đã giải quyết hoàn hảo cả phần lõi thuật toán Heuristic, Pipeline dữ liệu, Mạng nơ-ron ResNet và Học tăng cường MCTS. Hướng đi tiếp theo là: (1) Đóng gói giao diện Desktop Native hoàn chỉnh bằng khung công nghệ Tauri; (2) Mở rộng cơ chế tìm kiếm MCTS đa luồng (Root Parallelization) trên Web Worker; và (3) Triển khai chế độ đấu trực tuyến thời gian thực PvP/PvE qua giao thức WebSocket.

# Chapter 4

# Kết Luận & Bàn Giao Sản Phẩm Đồ Án

Đồ án Cơ sở đã đạt được toàn bộ các mục tiêu đề ra với chuẩn mực học thuật cao nhất:

1. Đã hoàn thành trọn vẹn cả 5 chặng đường nghiên cứu từ lý thuyết đến thực nghiệm.
2. Đã biên soạn và biên dịch thành công hệ thống **6 chuyên khảo tài liệu LaTeX** với tổng cộng hơn 130 trang nghiên cứu chuyên sâu.
3. Đã huấn luyện thành công mạng nơ-ron ResNet-6, công cụ tìm kiếm AlphaZero MCTS và vòng lặp tự chơi hoàn toàn trên phần cứng cá nhân.
4. Đã xuất khẩu mô hình chuẩn mở ONNX `caro_resnet.onnx` và `champion_ultimat`
5. Đã xây dựng hoàn chỉnh công cụ đấu đài tương tác trực tiếp `interactive_play.p` phục vụ kiểm thử thực chiến.

# Bibliography


[1] Silver, D., et al. (2017). *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. Science, 362(6419), 1140-1144.

[2] He, K., et al. (2016). *Deep Residual Learning for Image Recognition*. CVPR, 770-778.

[3] Wu, D. J. (2019). *Accelerating Self-Play Learning in Go*. arXiv:1902.10565.

[4] Loshchilov, I., & Hutter, F. (2017). *Decoupled Weight Decay Regularization*. ICLR.